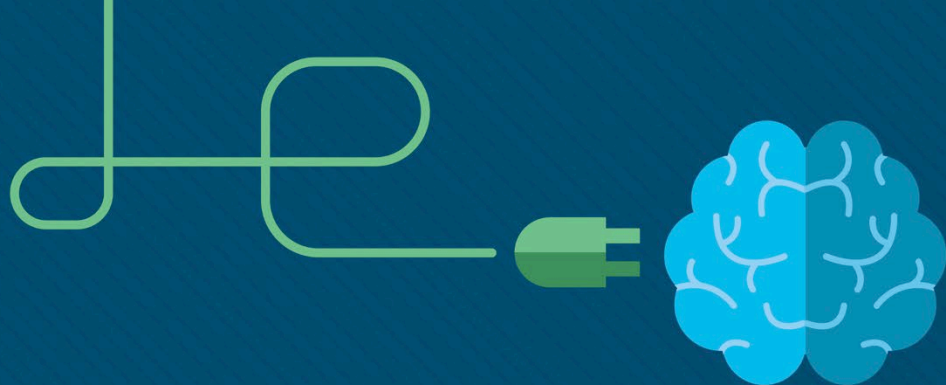




Week 11

RH134 Chapter 2: Scheduling Future Tasks
RH134 Chapter 4: Archive and Transfer Files
RH134 Chapter 10: Control the Boot Process



Scheduling Jobs

1 Introduction

- Scheduling tasks or executing commands at a particular time is inherently part of Linux administration. Some of the tasks (commands or shell scripts) which generally require scheduling are:
 - Generating thumbnails for websites
 - Computing intermediate data for processing
 - Performing backups
 - Scheduling operating system and application updates
 - Sending newsletter emails
 - Automatically checking websites for broken links
 - Housekeeping activities such as cleaning up directories that store log files
- There are a wide variety of commands that can be executed at a future time. However, keep in mind that the commands should be non-interactive, since the assumption is that there is no user to interact with when the commands execute. Generally speaking, four tools that is cron, anacron, at and batch will be extremely useful for scheduling tasks for execution at some later time. We will cover cron and anacron for this module.

2. Understanding Cron

- The cron utility consists of the following:
 - A time-based job scheduler daemon named crond. This daemon is used to execute repetitive tasks at a pre-determined interval.
 - A crontab (cron table) file that stores scheduled tasks (commonly referred to as jobs or cron jobs) to execute and is read by the crond daemon. Each user is permitted to have a personal crontab file by default but the administrator can disable this feature for security purposes.
 - A system crontab that allows the scheduling of system wide tasks such as log rotations, backups and system database updates.
 - The crontab command used to view and edit a user's crontab file.

3. System crontab and User crontab

- The system crontab allows scheduling of system wide tasks such as log rotations, backups and system database updates. Modifying the system crontab for scheduling tasks requires administrative privileges. Additional system crontab jobs that require finer control of their scheduling (verses daily, weekly, monthly tasks in /etc/crontab) and do not need environmental variables to be loaded before script execution are stored in the /etc/cron.d directory.
- The system crontab can execute commands either as root or as any other user.
- The output on the right is an example of a typical system crontab on a CentOS.

```
SHELL=/bin/bash
PATH=/sbin:/bin:/usr/sbin:/usr/bin
MAILTO=root

# For details see man 4 crontabs

# Example of job definition:
# .----- minute (0 - 59)
# | .----- hour (0 - 23)
# | | .----- day of month (1 - 31)
# | | | .----- month (1 - 12) OR jan,feb,mar,apr ...
# | | | | .---- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# | | | | |
# * * * * * user-name  command to be executed

30 08 10 06 * root /home/user1/bin/full-backup.sh
00 11,16 * * * root /home/user1/bin/check-db-status.sh
```

3. System crontab and User crontab (Cont'd)

- In addition to the system crontab maintained by the root user, each user can have a personal crontab file unless prevented by the administrator for security reasons. Like the system crontab file, these commands will run regardless of whether the user is logged into the system.
- The following is an example of a basic user crontab file:

```
#min hr mday month wday
#
17 0 * * 5 /usr/bin/banner "Weekend is here!" > /dev/console
```

- The user crontab is similar to the system crontab except for the “user” field which specifies the user whose privileges the cron job will run under in the system crontab. This is not included in the user crontab since all cron jobs listed in the user crontab will be run under the user’s privilege.

4. Understanding Crontab Entries

- crontab entries can be used to schedule tasks that can occur as frequently as once every minute to as infrequently as once a year. The format of a crontab entry is:

```
Minute Hour Day-of-month Month Day-of-week command
```

- The following table provides additional details regarding the values that can be placed in each of these fields:

Field	Values
Minute	Range 0 – 59
Hour	Range 0 – 23
Day-of-month	Range 1 – 31
Month	Range 1 – 12 (Jan-Dec)
Day-of-week	Range 0 – 7 (0 & 7 = Sunday)
Command	The command to be executed

4. Understanding Crontab Entries (Cont'd)

Example of cron entries showing various ways to indicate the values.

Execute the Full backup shell script on 10th June 08:30 AM.

```
30 08 10 06 * root /home/user1/bin/full-backup.sh
```

Executes incremental backup shell script at 11:00 and 16:00 on every day.

```
00 11,16 * * * root /home/user1/bin/incremental-backup.sh
```

checks the status of the database everyday (including weekends) during the working hours 9 a.m - 6 p.m

```
00 09-18 * * * root /home/user1/bin/check-db-status.sh
```

check the disk space every 10 minutes

```
*/10 * * * * root /home/user1/bin/check-disk-space.sh
```

4. Understanding Crontab Entries (Cont'd)

Variables in crontabs

- The crontab file can contain variables such as PATH, SHELL, MAILTO etc.

```
PATH=/usr/local/bin:/usr/local/sbin:/sbin:/bin:/usr/sbin:/usr/bin  
SHELL=/bin/bash  
MAILTO=databaseadmin
```

- The PATH variable is very important in the event that relative pathnames are used when specifying commands. For example, to execute the **date** command without including the PATH variable, the full path to the **date** command /bin/date must be specified.
- It is important to note that the crond utility does not read user initialization files when it executes commands. This means that creating variables like the PATH variable is very important.
- The MAILTO variable is important for any command that produces output when executed. There is no assumption that the user is logged in when the command runs, so the output of the command is sent to the user's email address by default. To specify a different email address, use the MAILTO variable.

4. Understanding Crontab Entries (Cont'd)

Crontab Keywords

The following table lists the special keywords that can be used to specify a time in place of the normal five table fields:

Entry	Description	Equivalent to
@reboot	Run once, at startup	
@yearly @annually	Run once a year	0 0 1 1 *
@monthly	Run once a month	0 0 1 * *
@weekly	Run once a week	0 0 * * 0
@daily @midnight	Run once a day	0 0 * * *
@hourly	Run once an hour	0 * * * *

5. Maintaining User crontab Files

- The crontab command is used for maintaining user crontab files. The user crontab files are stored in the /var/spool/cron directory, but should not be edited directly. In fact, regular users don't have any access to this directory, so a user must use the crontab command to display or edit his own crontab file. They don't have the rights to display their own or other users' crontab files.

```
sysadmin@localhost:~$ cat /var/spool/cron/crontabs/sysadmin
cat: /var/spool/cron/crontabs/sysadmin: Permission denied
```

- The crontab command will start a text editor program (vi by default) to allow the user to modify his own crontab file. To specify a different editor, set the EDITOR variable before executing the crontab -e command. For example,

```
sysadmin@localhost:~$ export EDITOR nano
sysadmin@localhost:~$
sysadmin@localhost:~$ crontab -e
```

- Users can view or remove all their crontab entries using the -l and -r options respectively. The root user can view a particular user's crontab file using the crontab -u <username> command.

```
GNU nano 2.2.6      File: /tmp/crontab.eq0yJk/crontab

# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#

[ Read 23 lines ]
^G Get Help  ^O WriteOut  ^R Read File ^Y Prev Page ^K Cut Text   ^C Cur Pos
^X Exit      ^J Justify   ^W Where Is ^V Next Page ^U UnCut Text^T To Spell
```

6. Maintaining System crontab Files

- The crond daemon reads the system crontab from the `/etc/crontab` file. In version 7 of the Red Hat Enterprise Linux, this file do not contain any entries. In previous version of Red Hat Enterprise Linux, there are entries in this file that direct the crond daemon to execute commands in the following directories:
 - `/etc/cron.hourly`
 - `/etc/cron.daily`
 - `/etc/cron.weekly`.
- In the current version, these hourly, daily and weekly jobs are run by `anacrontab`.
- The crond daemon also reads the files in `/etc/cron.d` directory and executes them. Software packages may create crontab files in the `/etc/cron.d` upon installation and remove them upon uninstallation.
- The crond daemon examines all the crontab files every minute and processes any jobs that it found needs to be executed.
- Upon any changes to the crontab files (eg, new cron entries added or old entries deleted), the crond daemon will reload them by checking the modification time of files. There is no need to restart the crond daemon if any crontab files are changed.

7. Controlling Access to cron

- Access to schedule cron jobs can be restricted to only authorized user accounts. This makes use of `/etc/cron.allow` (whitelist) and `/etc/cron.deny` (blacklist).
- These files contain a list of users, one line per user. The following rules apply regarding these files:
 - If both files do not exist, the default on current Ubuntu and other Debian-based systems, is to allow all users to use the `crontab` command.
 - If only the `cron.allow` file exists, then only the users listed in the file can execute the `crontab` command.
 - If only the `cron.deny` file exists, then all users listed in this file are denied the ability to execute the `crontab` command and all other users are allowed to use the `crontab` command.
 - If both the `cron.allow` file and the `cron.deny` file exist, the `cron.allow` file applies and the `cron.deny` file is ignored. This is typically a mistake made by the administrator as only one of these files should exist.
- Only the root user has the rights to modify these files.

```
sysadmin@localhost:~$ crontab -e
You (sysadmin) are not allowed to use this program (crontab)
See crontab(1) for more information
```

8. Anacron

- The **anacron** service is used to run commands periodically with a frequency specified in days (i.e. daily, weekly or monthly) on systems that do not run continuously, like a personal desktop or laptop computer. The primary purpose of the **anacron** command is to execute commands that were supposed to be executed, but weren't because the system was off.
- Similar to the crond daemon, the **anacron** command uses a configuration file: the `/etc/anacrontab` file. Entries in `anacrontab` have following format:

```
period    delay    job-identifier    command
```

- Consider the following entry in the `/etc/anacrontab` file. The script `nice run-parts /etc/cron.daily` will run 15 minutes after the system boots up if the last time the script ran was more than 1 day ago:

```
1        15        cron.daily    nice run-parts /etc/cron.daily
```

- The command **nice run-parts /etc/cron.daily** will execute all of the scripts in the `/etc/cron.daily` directory. Because it is run daily, this provides an easy way for software developers and system administrators to add a new daily task by adding a script to the `/etc/cron.daily` directory.

8. Anacron (Cont'd)

- The fields in the `/etc/anacrontab` is described in detailed below:

Field	Example	Significance
period	1	A numeric value that specifies the number of days A value of N executes every Nth day The <code>anacron</code> command will check if the job has run in the last N days based on the date (specific time not considered) in the job's timestamp field.
delay	15	Specified in minutes, this field indicates that the <code>anacron</code> command should wait this duration after the machine starts before executing the command.
job-identifier	<code>cron.daily</code>	Used to identify the job in <code>anacron</code> log messages. Also the name for the job's timestamp files which is stored in <code>/var/spool/anacron</code> directory.
command	<code>nice run-parts /etc/cron.daily</code>	The command or script that should be executed.

- Suppose this script runs on 03/08/2014, 9:30 a.m. and the system is then turned off. Then the server is booted on 03/11/2014 at noon; the backup script will be triggered 15 minutes later at 12:15 p.m. Additionally, the contents of the `/var/spool/anacron/cron.daily` file will be updated to reflect the last time this script was executed.

8. Anacron (Cont'd)

- Some of the key options of the `anacron` command are:

Option	Use
<code>-f</code>	Force the job to be run, irrespective of its time-stamp. This is typically used when running the <code>anacron</code> command manually.
<code>-u</code>	Only update the time-stamp of the job without running the job/script.
<code>-s</code>	Execute jobs sequentially, the next job will not be started unless the current job finishes.
<code>-n</code>	Run the job now, overrides the delay specified in the <code>/etc/anacrontab</code> file.

Difference between Crontab and Anacron

Cron	Anacron
For servers or other systems that run continuously.	For desktops and laptops not expected to be running 24 x 7.
<code>cron</code> daemon checks the <code>crontab</code> file every minute for jobs that need to run. If the system is not running at the time when the job is scheduled it does not run.	When the system is turned on, <code>anacron</code> checks if a scheduled job has been run in the last <code>N</code> days based on the date (specific time not considered) in the job's timestamp field.
Can run the same job multiple times a day.	Only run a job once a day.

Cron	Anacron
Minimum granularity to schedule a job is a minute.	Minimum granularity to schedule a job is only in days.
Cron job can be scheduled by a regular user (unless restricted by root).	Anacron can be used only by root.
<code>cron</code> daemon reads <code>/etc/crontab</code> .	<code>cron</code> daemon reads <code>/etc/anacrontab</code> . Note: on Ubuntu systems, the <code>/etc/anacrontab</code> file is only used if there is no <code>/etc/crontab</code> file. On Red Hat systems, both files can be used. Consult the <code>anacron</code> man page to determine how the <code>cron</code> daemon uses the <code>/etc/anacrontab</code> file on a particular system.

Archive Commands

9. The tar Command

- The tar command was originally used to create tarballs to be backed-up to magnetic tape storage. Tar is derived from the words “tape archive”.
- The tar command creates a single .tar file which most often contain of many files/directories. The primary purpose of the command is to merge multiple files / directories into a single file. However, it is capable of many operations which consists of the following:
 - Create: Make a new archive out of a series of files
 - Extract: Pull one or more files out of an archive
 - List: Show the contents of the archive without extracting
- The tar command accepts all three styles of options, (x, -x, --extract) as parameters.

9. The tar Command (Cont'd)

Create Mode

- Creating an archive with the tar command requires two named options:
 - -c : Create an archive
 - -f ARCHIVE: Use the ARCHIVE file. The argument ARCHIVE will be the name of the resulting archive file.

```
tar -c [-f ARCHIVE] [OPTIONS] [FILE...]
```

- All the remaining arguments, [FILE...], are considered as input file names either as a list of files, as a wildcard or both.
- For example, to create an archive of the Documents directory, use the create -c option and file -f option to specify a new file, followed by the directory to archive.

```
sysadmin@localhost:~$ tar -cf mytar.tar ~/Documents/  
tar: Removing leading `/' from member names  
sysadmin@localhost:~$ ls  
Desktop    Downloads  Pictures   Templates  mytar.tar  
Documents  Music      Public     Videos
```

9. The tar Command (Cont'd)

- Some additional options that can be used when using the tar command to create archive files are -v and -z.
 - -v : Verbosely list the files processed
 - -z : Compress (or decompress) the archive using the gzip command
- It is customary to add the .tar.gz extension to the compressed archive file name to easily identify the archive file within a directory. The next example below shows the use of the -z and -v options:

```
sysadmin@localhost:~$ tar -cvzf mytar.tar.gz ~/Documents/  
tar: Removing leading `/' from member names  
/home/sysadmin/Documents/  
/home/sysadmin/Documents/letters.txt  
/home/sysadmin/Documents/alpha-second.txt  
/home/sysadmin/Documents/spelling.txt  
/home/sysadmin/Documents/animals.txt  
/home/sysadmin/Documents/linux.txt  
/home/sysadmin/Documents/Work/  
/home/sysadmin/Documents/Work/.emptydir  
/home/sysadmin/Documents/words
```

- The tar command supports the following compression options:
 - -z : gzip
 - -j : bzip2
 - -J : xz

9. The tar Command (Cont'd)

List Mode

- Use the -t option to the tar command to view a list of a tar file contents:

- -t : List the files in the archive
- -f ARCHIVE : Operate on the given archive

```
tar -t [-f ARCHIVE] [OPTIONS]
```

- Even if the archive file is compressed, the correct compression option does not need to be specified to view a tar archive file. Only the file -f option and the list -t option are required to view the table of contents. Once again, note that the -f option is used last so that the file name can be specified as an argument to this option.
- Use the -v option if you wish to see the list in long format (similar to ls -l)

```
sysadmin@localhost:~$ tar -tf mytar.tar.gz
home/sysadmin/Documents/
home/sysadmin/Documents/letters.txt
home/sysadmin/Documents/alpha-second.txt
home/sysadmin/Documents/spelling.txt
home/sysadmin/Documents/animals.txt
home/sysadmin/Documents/linux.txt
home/sysadmin/Documents/Work/
home/sysadmin/Documents/Work/.emptydir
```

9. The tar Command (Cont'd)

Extract Mode

- To extract the files from the tar file, use the -x option.
 - -x : Extract files from an archive
 - -f ARCHIVE : Operate on the given archive

```
tar -x [-f ARCHIVE] [OPTIONS]
```

10. The dd Command

- The **dd** command is a utility for copying files or entire partitions at the bit level. This command has several useful features, including:
 - It can be used to clone or delete (wipe) entire disks or partitions.
 - It can be used to copy raw data to removable devices, such as USB drives and CDROMs.
 - It can backup and restore the MBR (Master Boot Record), a critical software component that is used to boot the system.
 - It can be used to create a file of a specific size that is filled with binary zeros, which can then be used as a swap file (virtual memory).
- The dd command uses special arguments to specify how it will work. The table on the right illustrates some of the more commonly used arguments:

<code>if=FILE</code>	The input file to be read.
<code>of=FILE</code>	The output file to be written.
<code>bs=SIZE</code>	The block size to be used. By default, the value is considered to be in bytes. Use the following suffixes to specify other units: <code>K</code> , <code>M</code> , <code>G</code> , and <code>T</code> for kilobytes, megabytes, gigabytes, and terabytes respectively.
<code>count=NUMBER</code>	The number of blocks to read from the input file.

10. The dd Command (Cont'd)

- In the following example, a file named /tmp/swapex is created with 500 "one-megabyte" size blocks of zeros:

```
sysadmin@localhost:~$ dd if=/dev/zero of=/tmp/swapex bs=1M count=500

500+0 records in
500+0 records out
524288000 bytes (524 MB, 500 MiB) copied, 64.0268 s, 8.2 MB/s

sysadmin@localhost:~$ ls -l /tmp/swapex
-rw-rw-r-- 1 sysadmin sysadmin 524288000 Nov 30 11:49 /tmp/swapex
```

- The following command copies the entire /dev/sda hard drive to the /dev/sdb

```
sysadmin@localhost:~$ dd if=/dev/sda of=/dev/sdb
```

- Finally, the following command can be used to make an .iso image backup of your CDROM or DVD device. It will take all of the data from the DVD /dev/dvd and stores the data into a local file called dvd.iso.

```
sysadmin@localhost:~$ dd if=/dev/dvd of=dvd.iso
```

10. The dd Command (Cont'd)

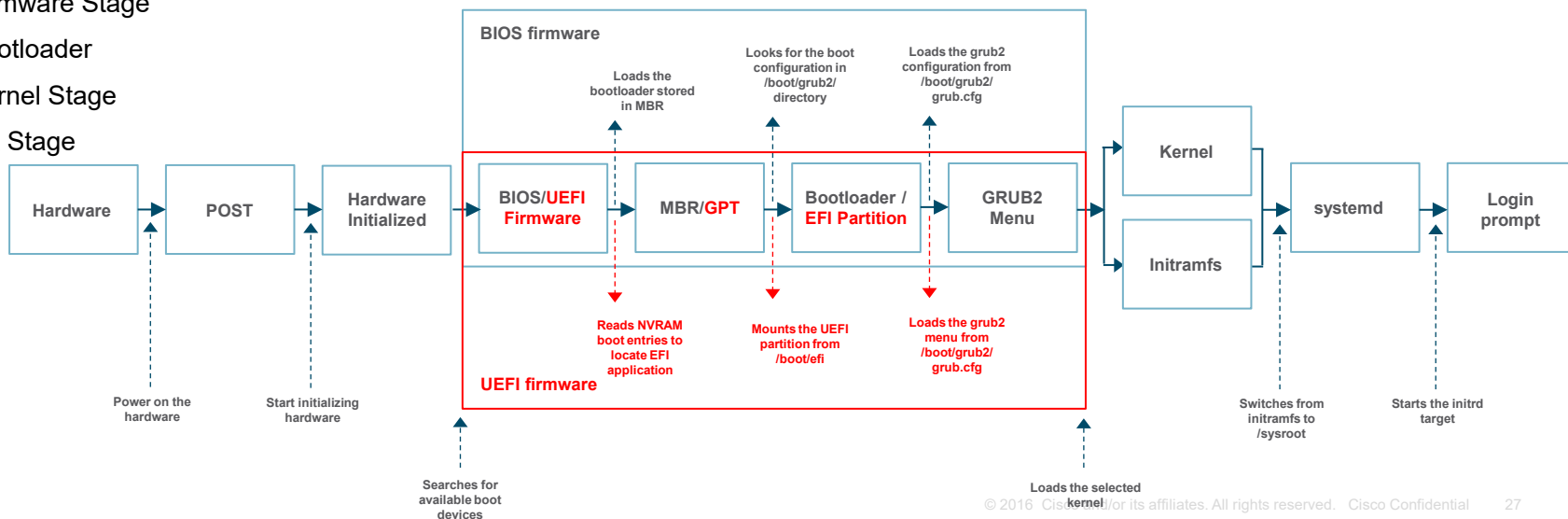
Device files are files used to refer to devices on the system, such as hard drives, CD-ROMS, and partitions. Device files are associated with file names that are stored in the `/dev` directory. The following information is provided to add clarity to the examples shown with the *dd* command:

<code>/dev/sda</code>	A device file that typically refers to the first hard drive on the system.
<code>/dev/sdb</code>	A device file that typically refers to the second hard drive on the system.
<code>/dev/dvd</code>	A device file that typically refers to the first DVD drive on the system.

Boot Process

11. Introduction

- Booting is the process of bringing the system from an *off* status to a running operating system. It is very important to understand this process, in order to be able to fix boot problems when they occur and for modifying system states called **systemd** devices, *targets*.
- The boot process takes place in four main stages, some of which are modified by administrators, while for the others, it is sufficient just to be aware of the actions taking place:
 1. Firmware Stage
 2. Bootloader
 3. Kernel Stage
 4. Init Stage



11.1 Firmware Stage

- The firmware stage is the first stage to take place after the computer is powered on. At this point, the computer has power but needs to start executing some software that will eventually get a full kernel booted.
- Most PC firmware is referred to as the **Basic Input/Output System (BIOS)**. The BIOS is stored on the motherboard in non-volatile memory such as Read Only Memory (ROM) or flash memory.
- The BIOS has three main jobs to perform as part of the first stage of the boot process:
 1. Execute a *power-on self test (POST)* in order to ensure the hardware of the system is functioning properly. The POST runs some basic functional checks on the CPU, memory, and peripherals so that obvious errors, such as missing memory chips or malfunctioning disk devices are found early in the boot cycle.
 2. Enumerate available hardware such as memory, disks, and USB devices.
 3. Find the proper boot drive from the available storage devices and load the **Master Boot Record (MBR)** from that device. The Master Boot Record is the first sector (or 512 bytes) of the disk.

11.1 Firmware Stage (Cont'd)

- Even on systems that have replaced the traditional BIOS with the **Unified Extensible Firmware Interface (UEFI)**, the system firmware is still often referred to as BIOS. Both are firmware, the hardware-embedded tiny operating systems that manage the interface between hardware and the OS that you will be booting. BIOS is considered to be dated because it is limited to 16-bit memory addressing, can only be a certain size and boots by reading the first sector of a drive. UEFI uses a special partition on a drive called an EFI System Partition to store the bootloader for an OS.
- The BIOS and UEFI firmware have become more sophisticated over time. However, on most boot-ups, they are there to perform the three main steps above in order to get to the bootloader stage.

11.2 Bootloader Stage

- The Master Boot Record contains a partition table and a very small amount of executable code called the *first stage bootloader* whose purpose is to load the more feature-rich *second stage bootloader*. The bootloader will perform several operations, but the primary task is to load the Linux kernel into memory and execute it.
- If a bootloader is able to fit entirely into the first 512 bytes of the Master Boot Record and to start the operating system with this limited amount of code, it is considered a single stage bootloader. A second stage is necessary when the code needed to initialize the system properly and start the operating system does not fit in the MBR. In this case, the first stage bootloader is there to point to the second stage bootloader, which can be larger and will be the bootloader that actually starts the operating system. Once that has occurred, the kernel takes over booting the system.
- The most common bootloader used on machines is the **Grand Unified Bootloader (GRUB)**. The latest version of GRUB supports booting Linux from a system using UEFI, interactive editing of the menu during bootup, and much more.
- UEFI systems give the firmware stage a lot more memory and capabilities so that it can handle much larger and more complicated hardware.

11.2 Bootloader Stage (Cont'd)

- It is also possible to boot off the network through the **Preboot Execution Environment (PXE)**. In the PXE system, a compatible motherboard and network card contain enough intelligence to acquire an address from the network and use the **Trivial File Transfer Protocol (TFTP)** to download a special bootloader from a server.
- As the bootloader is just some software that gets a kernel to run, it is possible to boot multiple operating systems at different times off of one computer in a process known as dual or multi-booting. The kernel that the bootloader is trying to run could be a Linux kernel, it could be a Microsoft Windows image, or it could be a bootable CD.
- The bootloader can also pass parameters to the kernel, such as to boot into a maintenance mode or to enable or disable certain hardware. This is done by manipulating the bootloader configuration. GRUB provides a reasonably powerful command line interface that lets an administrator make changes to the kernel before it boots without requiring that the configuration be written to disk.
- The bootloader then loads the kernel from disk into memory and transfers control over. The system is now running Linux and may finish booting.

11.3 Kernel Stage

- Now that the bootloader has loaded the kernel into memory, there is much work to be done before programs can be loaded. The kernel must initialize any hardware drivers and get the root / filesystem mounted for the next stage. These two tasks are actually quite complicated because the facilities provided by the BIOS to access the hardware are quite limited. Therefore, the kernel must boot the system in several phases.
- The kernel itself is laid out much like a regular executable except that it must be self-contained. Shared libraries are not available at this point in the boot process so the kernel itself is statically linked, meaning that the kernel can only rely on information that was already compiled in the kernel at boot time and not on information in the shared libraries. This kernel typically lives in the `/boot` partition which, on most hardware, is in a separate partition that's kept close to the beginning of the hard drive. This location is important for some BIOS and bootloader combinations that can only access the first 1024 cylinders of the disk.
- The `/boot` directory, alternatively the `/boot/efi` directory, is one of the most important directories in the File Hierarchy Standard (FHS). Most modern systems use the UEFI standard for specifying the exact location where the computer looks for the files it needs to start up. You may hear the term *bootstrap* used in this context, which simply signifies the method for the computer BIOS to locate and load the first part of the operating system when the computer powers up.

11.3 Kernel Stage (Cont'd)

- The Linux kernel must mount the root filesystem in order to get to the next step and to make the system useful. However, it is possible that the root filesystem exists on a device that the kernel does not know how to support. The solution to this is the initial RAM disk `initrd`. The kernel drivers necessary to continue the boot are bundled into the filesystem that is stored beside the kernel itself in `/boot`. The kernel boots, mounts the `initrd`, loads the drivers inside, and then remounts the real root filesystem using the new drivers. This allows drivers to be added to the kernel easily and for the kernel to mount the root filesystem over almost any storage hardware—even if it's via a network.
- As the kernel is booting, it is able to initialize recognized hardware and make detected devices available to the rest of the operating system.
- The kernel's final job is to start the first process on the system. The first process will normally have a process id (PID) of 1; on a System V system, the name of this process is `init`.
- On **systemd** systems, the `/sbin/init` file is a symbolic link to the `/usr/lib/systemd/systemd` file for purposes of compatibility.

11.3 Kernel Stage (Cont'd)

- To show the first process, `init`, execute the `ps` command using the following non-BSD options, which will show all processes running on a system in a hierarchical format:

```
[sysadmin@centos ~]$ ps -ejH
```

PID	PGID	SID	TTY	TIME	CMD
2	0	0	?	00:00:00	kthreadd
4	0	0	?	00:00:00	kworker/0:0H
6	0	0	?	00:00:00	ksoftirqd/0
7	0	0	?	00:00:00	migration/0
8	0	0	?	00:00:00	rcu_bh
9	0	0	?	00:00:00	rcu_sched
10	0	0	?	00:00:00	lru-add-drain
11	0	0	?	00:00:00	watchdog/0
13	0	0	?	00:00:00	kdevtmpfs
14	0	0	?	00:00:00	netns
15	0	0	?	00:00:00	khungtaskd
1	1	1	?	00:00:00	systemd
407	407	407	?	00:00:00	systemd-journal
430	430	430	?	00:00:00	lvmetad
438	438	438	?	00:00:00	systemd-udev
626	626	626	?	00:00:00	auditd

- The first process is responsible for much of the operation of the system from here on in. For our purposes, we use `/sbin/init` for consistency, keeping in mind that a different system may be running `systemd` instead of System V for process management.
- In some cases, such as embedded hardware, the `init` process could be a shell or a specialized daemon. In any case, this first process will start the daemons that the rest of the system will use. This process will be the parent process for any process that otherwise is missing a parent. This process persists for the life of the system.

11.4 The init Stage

- The init stage finishes booting the system. The first process of the operating system (also called `init`) is started. The init process has three important responsibilities:
 1. Continue the booting process to get services running, login screens displaying, and consoles listening.
 2. Start all other system processes.
 3. Adopt any process that detaches from its parent.
- Until recently, this process followed a design that was established with the release of System V of Unix, which is sometimes referred to as SysVinit. The actual process that is executed is the `init` process. Recently, other programs have emerged to compete with and replace the traditional `init` process with Upstart and systemd.
- If the system uses the traditional `init` program, then the `/etc/inittab` file is used to determine what scripts will be executed to start the services that will be available on the system.
- The `inittab` file points to other scripts that do the work, usually stored in the `/etc/init.d` directory. There will be more discussion of these scripts later, but in general, each service the system will run has a script that can start, stop, and restart a service, and `init` will cause each of these to be run in turn as needed to get the system to the desired state.

11.4 The init Stage (Cont'd)

- If the traditional `init` has been replaced with Upstart, the scripts in the `/etc/init` directory are used to complete system initialization.
- If the traditional `init` has been replaced with Systemd, then the files in the `/etc/systemd` directory are used for starting up and running the system.
- Even if your system is using Systemd or Upstart as a replacement for the traditional `init` process, both replacements use an executable named `init` with the `/sbin/init` path name. This is to maintain compatibility with many legacy processes. So, while some of the behavior of Systemd and Upstart will be different, they have some features that are similar to the traditional `init` process.

Bootloaders

12. Introduction

- The boot process for a computer goes through several distinct steps, starting from no power being applied and completing with a fully-running operating system. The bootloader is a critical part of the system startup process, essentially taking the computer from its reading of firmware for a boot device, to the point at which control is transferred to the system initialization scheme for the operating system that is chosen to run the computer.
- The two bootloaders most commonly used with Linux are the **Grand Unified Bootloader (GRUB)**, or the **Grand Unified Bootloader 2 (GRUB 2)**, though the latter is the one that is used most often. Previously, the **Linux Loader (LILO)** was the one most implemented, but it has fallen out of wide usage and is present mostly on older or legacy systems.
- GRUB is a GNU project with the goal of making a bootloader that can boot many different kernels on a variety of systems. GRUB can read filesystems and dynamically learn about the hardware. An administrator can boot into GRUB, query the disks, then discover and boot a kernel image. Also, rather than replacing the bootloader installation on disk each time a configuration change is made, the bootloader automatically reads configuration files.

12. Introduction (Cont'd)

- As a dynamic bootloader, GRUB also offers a command line interface that can override the settings from the configuration file. The most typical use of this is to boot into a rescue mode by passing extra kernel parameters or to specify limitations such as those used for memory troubleshooting.
- When a distribution is installed, a bootloader is usually installed automatically, reflecting the hardware particulars for the system. In most cases, the default bootloader will be either **GRUB Legacy** (version 0-0.9) or GRUB 2. Typical customizations at the point of install are to add extra operating systems for multi-booting, changes to go along with kernel upgrades, and booting into maintenance mode.

13. GRUB 2

- The new GRUB, Grand Unified Bootloader 2, offers several advantages:
 - Dynamically loaded modules
 - Non-ASCII character support
 - Ability to boot from partitions inside of Logical Volume Management (LVM) or RAID devices
 - Ability to work with architectures that don't have a PC BIOS
- When a GRUB 2 system is powered on, the first stage loader `boot.img` is either read from the first 440 bytes of the Master Boot Record (MBR), or alternatively read from the Partition Boot Record (PBR), (like a Master Boot Record for a partition).
- From the first stage loader, the location of the second stage loader is found. The second stage loader `diskboot.img` is used as the first sector of the third stage loader, and this points directly to the third stage.
- The third stage loader `core.img` uncompresses itself, accesses its own GRUB 2 kernel and filesystem drivers and attempts to access the actual root filesystem's `/boot/grub` directory.
- If it can access the `/boot/grub` directory on disk, it then loads the appropriate fourth stage module `normal.mod` which then reads the `/boot/grub/grub.cfg` file and presents either a GUI for GRUB 2 or the CLI menu so the user can choose an image to boot.

14. GRUB 2 Configuration

With GRUB Legacy, the files for GRUB were kept in the `/boot/grub` directory. Now, GRUB 2 uses several locations for its files:

File Location	Details
<code>/boot/grub2</code>	• Contains many module files. • Contains the <code>grub.cfg</code> file, a configuration file which should not be edited manually. • Contains the <code>grubenv</code> file, a configuration file which should not be edited manually
<code>/etc/default/grub</code>	• Contains the entries for creating a default boot entry; typically where customization by administrators will be performed.
<code>/etc/grub.d</code>	• Contains scripts that are run by GRUB 2. • Scripts can be customized or added to this directory. • Contains the following executable scripts by default <code>00_header</code>, <code>10_linux</code>, <code>20_linux_xen</code>, <code>30_os-prober</code>, <code>40_custom</code>, <code>41_custom</code>, <code>90_persistent</code>
<code>/usr/lib/grub</code>	• Stores a couple of library files
<code>/sbin/grub2-mkconfig</code>	• Command used to regenerate the <code>/boot/grub2/grub.cfg</code> file

14. GRUB 2 Configuration (Cont'd)

This following demonstrates a typical /etc/default/grub file:

```
[sysadmin@centos ~]$ cat /etc/default/grub
GRUB_TIMEOUT=5
GRUB_DISTRIBUTOR="$(sed 's, release .*$,,g' /etc/system-release)"
GRUB_DEFAULT=saved
GRUB_DISABLE_SUBMENU=true
GRUB_TERMINAL_OUTPUT="console"
GRUB_CMDLINE_LINUX="rhgb quiet"
GRUB_DISABLE_RECOVERY="true"
[sysadmin@centos ~]$
```

The new GRUB 2 allows the user to choose a new default menu item. By adding the following settings in the /etc/default/grub file and regenerating the configuration file, whatever title the user chooses from the menu becomes “saved” and will be the default the next time the menu is displayed:

```
GRUB_DEFAULT=saved
GRUB_SAVEDEFAULT=true
```

After modifying the /etc/default/grub file, execute either the **update-grub** command (for Ubuntu-based systems) or the **grub2-mkconfig** command (for Red Hat-based systems) to update the actual configuration file used by GRUB 2 at boot time.

14. GRUB 2 Configuration (Cont'd)

The GRUB_TIMEOUT value found in the /etc/default/grub file establishes how many seconds to wait before the system is automatically booted to the default menu entry. If the user presses any key except the Enter key, then the timeout is aborted, and the user can select a menu entry manually.

If GRUB 2 needs to be installed or reinstalled, then an administrator would execute:

```
[root@centos /root]# /sbin/grub2-install /dev/sda
```

After installing GRUB 2, the configuration file needs to be generated for the first time. In a Fedora-based distribution, an administrator would execute:

```
[root@centos /root]# grub2-mkconfig -o /boot/grub2/grub.cfg
```

15. How to: Recover Root Password

- Step 1: Reboot the system
- Step 2: Interrupt the boot loader countdown by pressing any key
- Step 3: Move the cursor to the entry that needs to be booted
- Step 4: Press **e** to edit the selected entry 

```
GRUB version 2.06

load_video
set gfxpayload=keep
insmod gzio
linux ($root)/boot/vmlinuz-5.14.0-435.el9.x86_64 root=UUID=2709c584-4430-47\
1c-9c00-7761e8b86dd7 ro crashkernel=1G-4G:192M,4G-64G:256M,64G-:512M resume\
=UUID=52e2cccd-b856-4e4e-a16f-9e65d6d110f5 rhgb quiet
initrd ($root)/boot/initramfs-5.14.0-435.el9.x86_64.img $tuned_initrd

Minimum Emacs-like screen editing is supported. TAB lists
completions. Press Ctrl-x or F10 to boot, Ctrl-c or F2 for
a command-line or ESC to discard edits and return to the GRUB menu.
```

15. How to: Recover Root Password (Cont'd)

- Step 5: Move the cursor to the kernel command line (the line that starts with linux)
- Step 6: Append **rd.break** (this will break just before control is handed from the initramfs to the actual system)
- Step 7: Press Ctrl-X to boot with the changes

```
GRUB version 2.06

load_video
set gfxpayload=keep
insmod gzio
linux ($root)/boot/vmlinuz-5.14.0-435.el9.x86_64 root=UUID=2709c584-4430-47\
1c-9c00-7761e8b86dd7 ro crashkernel=1G-4G:192M,4G-64G:256M,64G-:512M resume\
=UUID=52e2cccd-b856-4e4e-a16f-9e65d6d110f5 rhgb quiet rd.break
initrd ($root)/boot/initramfs-5.14.0-435.el9.x86_64.img $tuned_initrd_

Minimum Emacs-like screen editing is supported. TAB lists
completions. Press Ctrl-x or F10 to boot, Ctrl-c or F2 for
a command-line or ESC to discard edits and return to the GRUB menu.
```

15. How to: Recover Root Password (Cont'd)

- Step 8: Remount /sysroot as read-write
- Step 9: Switch into a chroot jail, where /sysroot is treated as the root of the file system tree
- Step 10: Set a new root password
- Step 11: Make sure that all unlabelled files (including /etc/shadow at this point) get relabeled during boot.
- Step 12: Type exit twice. The first will exit the chroot jail, and the second will exit the initramfs debug shell.

```
Generating "/run/initramfs/rdsosreport.txt"
```

```
Entering emergency mode. Exit the shell to continue.  
Type "journalctl" to view system logs.  
You might want to save "/run/initramfs/rdsosreport.txt" to a USB stick or /boot  
after mounting them and attach it to a bug report.
```

```
switch_root:/# mount -o remount,rw /sysroot
```

```
switch_root:/# chroot /sysroot
```

```
sh-5.1# passwd
```

```
Changing password for user root.
```

```
New password:
```

```
BAD PASSWORD: The password fails the dictionary check - it is based on a dictionary word
```

```
Retype new password:
```

```
passwd: all authentication tokens updated successfully.
```

```
sh-5.1# touch /.autorelabel
```

```
sh-5.1# exit
```

```
exit
```

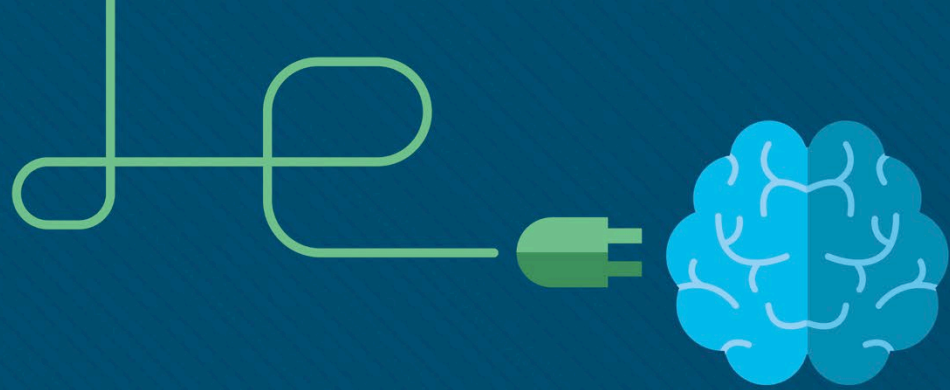
```
switch_root:/# exit_
```

Step 8: Remount
/sysroot as read-write

Step 9: Switch into a
chroot jail with
/sysroot being the root

Step 10: Set a
new root
password

Step 11: Relabel
unlabelled files



Lab

RH134 Chapter 4.7: Lab Archive and Transfer Files

RH134 Chapter 10.7: Lab Control the Boot Process



Practice Exercises

1 Scheduling Jobs with Cron & Anacron

- List all scheduled cron jobs for the current user.
- Schedule a cron job to run a script `/home/user/backup.sh` every day at 2 AM.
- Create an anacron job that runs a script `/home/user/cleanup.sh` every 3 days.
- View the system-wide crontab file. Where are additional cron jobs stored?

2 Archiving & Transferring Files

- Create a tar archive of `/home/user/documents` and name it `backup.tar`.
- Compress `backup.tar` using `gzip`. What is the final filename?
- Extract the contents of `backup.tar.gz` into `/home/user/recovered`.

3 Understanding the Boot Process

- Identify the current system boot time using `who`.
- View the GRUB configuration file location.
- List the last 5 boot logs from the system journal.

4 Recovering Root Password

- What key should be pressed to interrupt the bootloader during startup?
- Which command allows modifying the kernel parameters to reset the root password?
- What must be done after resetting the root password to ensure proper relabeling?